

Advanced Programming

Project assignment – 6th stage

Learning Mode

Undo/Redo: Command and Memento patterns

Sounds

Stage 6

- All students must update the project and *checkout* the `main` *branch*
- The student responsible for this task should start by creating a new *branch* from the `main` *branch*
 - *Branch* name: **stg6extras**
 - The student who creates the *branch* should *push* it to *GitHub* to make it available to the rest of the group
- The remaining students should update the project and then *checkout* the **stg6extras** *branch*

Stage 6

- The deadline to perform the *Pull Request* and the *Squash and Merge* is 23:59 on the day before the lesson during the week of May 19-23
 - Worker-students should complete this stage by 2025.05.24
- The *Squash and Merge* into the *main* branch must be performed on the *GitHub* website
 - The *branch* must not be deleted after the *Merge* is completed

Stage 6

- In this stage, the following features are to be implemented in the chess game:
 - Learning mode
 - Display of possible moves when a piece is selected
 - Undo and redo functionality to allow users to go back and replay previous moves
 - Accessibility through audio feedback
 - Moves must be announced using audio descriptions to assist users with visual impairments or to improve overall accessibility

Learning mode

- When the "*Learning*" menu option is activated, the related menu items should become available:
 - *Show possible moves*
 - *Undo*
 - *Redo*

Learning mode

- When a piece is selected:
 - Retrieve from the data model the list of valid moves for the selected piece
 - Highlight on the board the squares to which the piece can legally move
 - Once the move is completed or a different piece is selected, the previous highlights should be cleared

Learning mode

- The *undo* and *redo* functionalities must be implemented using one of the following design patterns covered in the theoretical classes:
 - *Command*
 - *Memento*
- The classes involved in the implementation of the chosen pattern should be organized within a new subpackage of `pt.isec.pa.chess.model`
- The `ChessGameManager` class, which serves as the *Facade* for the data model, must also act as the *Facade* for the chosen design pattern, being responsible for forwarding *undo* and *redo* requests to the corresponding pattern components

Audio feedback

- To raise students' awareness of the importance of designing features accessible to users with interaction difficulties, the game must include audio descriptions of the moves performed by the players
- Each time a move is executed, a sound must be played announcing:
 - the name of the piece moved,
 - the origin cell,
 - the destination cell,
 - and, when applicable, whether a piece was captured or if the king is in check

Audio feedback

- The base project already includes sample sound files that may be used for this purpose
 - Additional sounds may be added if necessary
- A UI element must be included to allow users to toggle sound on or off